

Minimum Length of String After Deleting Similar Ends

Company: Goldman Sachs

Year: Not specified

Difficulty: Medium

Topic: Two Pointers, Strings

Source: LeetCode 1750 – Minimum Length of String After Deleting Similar Ends

[LeetCode 1750 – Minimum Length of String After Deleting Similar Ends](#)

Problem Description

You are given a string s containing only the characters:

a, b, c

You can repeatedly perform the following operation:

1. Choose a non-empty prefix where all characters are the same.
2. Choose a non-empty suffix where all characters are the same.
3. The prefix and suffix must not overlap.
4. The prefix and suffix must contain the **same character**.
5. Delete both the prefix and suffix.

Your task is to find the **minimum possible length** of the string after performing this operation any number of times.

Input Format

- The first line contains a string s consisting only of the lowercase characters a, b, and c.

Output Format

- Print a single integer — the minimum possible length of s after repeatedly deleting matching prefixes and suffixes.

Constraints

- $1 \leq s.length \leq 10^5$

- s only consists of the characters a, b, and c.

Example 1

Input:

s = "ca"

Output:

2

Explanation

The first character is c and the last character is a.

Since they are different, no operation can be performed.

Therefore, the string remains:

"ca"

and the minimum length is:

2

Example 2

Input:

s = "cabaabac"

Output:

0

Explanation

We can repeatedly remove matching characters from both ends:

cabaabac

↓

abaaba

↓

baab

↓

aa

↓

""

Therefore, the minimum length is:

0

Example 3

Input:

s = "aabccabba"

Output:

3

Explanation

First remove:

prefix = "aa"

suffix = "a"

The string becomes:

bccabb

Then remove:

prefix = "b"

suffix = "bb"

The string becomes:

cca

Now the first character is c and the last character is a, so no more operations are possible.

Therefore:

3

is the answer.

Approach to Solve This Problem

We can solve this problem using **Two Pointers**.

Use two pointers:

left → beginning of the string

right → end of the string

The important observation is:

We can perform an operation only when $s[\text{left}] == s[\text{right}]$.

If the characters at both ends are different, we cannot remove anything else, so the answer is simply:

$\text{right} - \text{left} + 1$

If they are the same, we can remove **all consecutive occurrences** of that character from both ends.

For example:

$s = \text{"aabccabba"}$

Initially:

$\text{left} = 0 \rightarrow \text{'a'}$

$\text{right} = 8 \rightarrow \text{'a'}$

Both are a, so we can remove the as from both ends.

The left side contains:

aa

and the right side contains:

a

After removing them:

bccabb

Now the ends are:

b ... b

Again, they are equal, so remove all consecutive bs from both ends.

Continue until:

- left > right, meaning the whole string was removed, or
- s[left] != s[right], meaning no more operations are possible.

Java Solution

```
import java.util.*;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        String s = sc.nextLine();
```

```
int left = 0;

int right = s.length() - 1;

while (left < right && s.charAt(left) == s.charAt(right)) {

    char ch = s.charAt(left);

    // Remove same characters from the left
    while (left <= right && s.charAt(left) == ch) {
        left++;
    }

    // Remove same characters from the right
    while (left <= right && s.charAt(right) == ch) {
        right--;
    }
}

System.out.println(right - left + 1);

}
```

Solution:

1. **Read the input:** main() reads the string s, matching the Input Format.
2. **Initialize two pointers:** Set left = 0 to point to the first character and right = s.length() - 1 to point to the last character.

3. **Check the characters at both ends:** While $\text{left} < \text{right}$ and $s[\text{left}] == s[\text{right}]$, an operation is possible because the prefix and suffix can contain the same character.
4. **Store the current character:** Store $s[\text{left}]$ in ch . We need to remove all consecutive occurrences of this character from both ends.
5. **Remove characters from the left:** Move left forward while the current character is equal to ch . This removes the matching prefix.
6. **Remove characters from the right:** Move right backward while the current character is equal to ch . This removes the matching suffix.
7. **Repeat the process:** Continue checking the new left and right characters. If they are equal, remove their matching groups again. If they are different, no further operation is possible.
8. **Calculate the remaining length:** When the process stops, the remaining characters are from index left to index right . Therefore, the remaining length is $\text{right} - \text{left} + 1$.
9. **Print the result:** Print $\text{right} - \text{left} + 1$, matching the Output Format. If $\text{left} > \text{right}$, the expression gives 0, meaning the entire string was removed.

Complexity Analysis

- **Time Complexity:** $O(N)$ — each character is removed from either the left or right side at most once.
- **Space Complexity:** $O(1)$ — only two pointers and one character variable are used.